How to Troubleshoot Crashing Kubernetes Pods

Understanding how to debug a crashing pod is the most common daily task for any junior DevOps engineer. This guide gives you the exact steps and commands to find and fix the issue before anyone else notices.

What you'll learn:

1. 1. Reading Past Logs (The --previous Flag)
2. 2. Checking Pod Events with Describe
3. 3. Finding Out of Memory (OOM) Crashes
4. 4. Watching Live Logs on Restart

Aman Raj Singh
Cloud Operations Engineer @ iManage
linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

1. Reading Past Logs (The --previous Flag)

When a pod crashes and restarts, running the standard log command only shows the logs of the new running container. To see the actual error that caused the crash in the first place, you must tell Kubernetes to fetch the logs of the previous, terminated container.

```
$kubectl logs my-app-pod --previous
```

2

TIP 2 OF 4

2. Checking Pod Events with Describe

Sometimes a pod crashes before it can even start writing logs, usually due to bad configuration or network issues. Using the describe command shows you a chronological list of system events, like image pulling, scheduling, and container creation errors.

```
$kubectl describe pod my-app-pod
```

3

TIP 3 OF 4

3. Finding Out of Memory (OOM) Crashes

If your application tries to use more RAM than Kubernetes allows, the system will instantly kill it with an 'OOMKilled' status. You can check the termination reason directly in the pod's status metadata to see if you need to allocate more memory.

```
$kubectl get pod my-app-pod -o jsonpath='{.status.containerStatuses[*].state.terminated.reason}'
```

4

TIP 4 OF 4

4. Watching Live Logs on Restart

If your pod is crashing repeatedly in a loop, you can attach your terminal to the log stream to watch the crash happen live. This helps you catch the exact moment the application initializes, connects to the database, and fails.

```
$kubectl logs -f my-app-pod
```

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

+ A restart count greater than 0 means your pod has crashed at least once.
+ CrashLoopBackOff means Kubernetes is waiting longer and longer before trying to restart your broken pod.
+ The --previous flag is your best friend for finding the root cause of a past crash.
+ Always check the 'Events' section at the bottom of the describe command output.
+ An exit code of 137 almost always means your pod was killed for using too much memory (OOMKilled).

Watch Out For

! Forgetting to use the --previous flag, which results in looking at empty logs of the newly restarted pod.
! Assuming all crashes are code bugs, when they are often just incorrect environment variables or secret keys.
! Not setting memory limits, which can cause a single crashing pod to slow down the entire server.
! Misspelling the container image name and wondering why the pod is stuck in ImagePullBackOff.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mramanrajsingh